ComicCraft – AI-Powered Story to Comic Generator

Naan Mudhalvan Project Testing Documentation

Project Name: ComicCraft
Project Type: AI-Powered Story-to-Comic Application
Project Domain: Artificial Intelligence, Natural Language Processing, and Visual Storytelling
Documentation Folder: 6_Project_Testing
Documentation File: README.md
Primary Workflow: Story Input → Story Processing → Comic Panel Generation → Final Comic

---

1. Introduction

ComicCraft is an AI-assisted application designed to transform written stories into visually engaging comic panels. The system accepts a story from the user, analyzes its narrative, divides it into scenes, prepares comic panel descriptions, generates or retrieves illustrations, integrates dialogue and captions, and assembles the panels into a final comic.

Software testing is an important part of the ComicCraft development process. Testing helps verify that each module performs its intended function and that the complete application produces a coherent comic from the original story.

This document describes the testing objectives, scope, methodology, test environment, test cases, expected results, error-handling checks, integration tests, acceptance criteria, and testing deliverables for ComicCraft.

Note: The test cases in this document describe the planned verification process. Actual test results, pass/fail status, screenshots, and defect details should be recorded after executing the tests against the implemented application.

2. Purpose of Testing

The purpose of testing ComicCraft is to verify that the application correctly transforms story text into an ordered set of comic panels and produces the expected final output.

Testing is intended to identify defects in input validation, story processing, scene segmentation, panel generation, image handling, dialogue integration, comic layout, and output generation.

Testing also helps determine whether the application handles invalid input, incomplete data, unavailable services, and other failures in a controlled manner.

3. Testing Objectives

The main objectives are:

- Verify that users can enter and submit valid stories.
- Confirm that empty or invalid story input is handled appropriately.
- Validate the processing of story content.
- Check whether scenes are generated in a logical sequence.
- Verify that each scene can be converted into a comic panel description.
- Check whether generated or retrieved images are associated with the correct panels.
- Validate the association of dialogue and captions with scenes.
- Verify that comic panels are arranged in the correct order.
- Confirm that the final comic preview displays correctly.
- Test error handling and recovery.

- Verify integration between the application's modules.
- Assess the usability and reliability of the story-to-comic workflow.
- Record defects and confirm that corrections work as intended.

4. Problem Statement

Creating a comic manually requires story planning, scene selection, illustration, dialogue placement, and panel arrangement. These activities can be time-consuming and require artistic or technical skills.

ComicCraft aims to simplify this process through an automated story-to-comic workflow.

However, automated processing may introduce problems such as incorrect scene interpretation, missing panels, inconsistent illustrations, misplaced dialogue, and layout errors. Testing is therefore required to verify that the application produces a readable and logically ordered comic.

5. System Under Test

The ComicCraft application is tested as a sequence of connected modules.

5.1 Story Input Module

Accepts written story text from the user.

Testing verifies input acceptance, empty-input handling, supported length limits, and submission behavior.

5.2 Input Validation Module

Checks whether the story meets the application's input requirements.

Testing verifies that invalid submissions are rejected appropriately and that valid stories can continue through the workflow.

5.3 Story Processing Module

Analyzes the story and extracts relevant narrative information, such as characters, events, locations, and dialogue, according to the implemented processing method.

Testing checks whether the resulting information